```
## markdown
# Phase 3 - Business KPI Calculations
StreamFlix Content Analytics Project

Name: [Mark Maxwel Louis]

Here I calculate the 10 KPIs from the brief, plus a look at subscriber segments,
churned subscribers, and content spend efficiency. I also attempt the bonus cohort
retention question at the end.

## code
import pandas as pd

subscribers = pd.read_csv('subscribers_clean.csv')
titles = pd.read_csv('titles_clean.csv')
watch_history = pd.read_csv('watch_history_clean.csv')
watchlist = pd.read_csv('watchlist_clean.csv')
ratings = pd.read_csv('ratings_clean.csv')

subscribers['signup_date'] = pd.to_datetime(subscribers['signup_date'])
subscribers['churn_date'] = pd.to_datetime(subscribers['churn_date'])
watch_history['watch_date'] = pd.to_datetime(watch_history['watch_date'])

wh_titles = watch_history.merge(titles, on='title_id', how='left')
print('Data loaded.')

## markdown
## KPI 1: Total Watch Hours

## code
total_watch_hours = watch_history['watch_duration_min'].sum() / 60
print('Total Watch Hours:', round(total_watch_hours))

## markdown
## KPI 2: Active Rate (target: above 70%)

## code
active_subscribers = subscribers[subscribers['is_active'] == True]
inactive_subscribers = subscribers[subscribers['is_active'] == False]

active_rate = len(active_subscribers) / len(subscribers) * 100
print('Active Rate:', round(active_rate, 1), '%')

if active_rate > 70:
    print('This meets the target of above 70%')
else:
    print('This does NOT meet the target of above 70%')

## markdown
## KPI 3: Churn Rate (target: below 30%)

## code
churn_rate = len(inactive_subscribers) / len(subscribers) * 100
print('Churn Rate:', round(churn_rate, 1), '%')

if churn_rate < 30:
    print('This meets the target of below 30%')
else:
    print('This does NOT meet the target of below 30%')

## markdown
## KPI 4: Average Completion Rate (target: above 60%)

## code
avg_completion = watch_history['completion_pct'].mean()
print('Average Completion Rate:', round(avg_completion, 1), '%')

if avg_completion > 60:
    print('This meets the target of above 60%')
else:
    print('This does NOT meet the target of above 60%')

## markdown
## KPI 5: Monthly Recurring Revenue (MRR)

## code
mrr = active_subscribers['monthly_price_usd'].sum()
```

```
print('MRR: $', round(mrr, 2))

## markdown
## KPI 6: ARPU (Average Revenue Per active User)

## code
arpu = mrr / len(active_subscribers)
print('ARPU: $', round(arpu, 2))

## markdown
## KPI 7: Average Watch Hours per Active Subscriber

## code
avg_hours_per_subscriber = total_watch_hours / len(active_subscribers)
print('Average Watch Hours per Active Subscriber:', round(avg_hours_per_subscriber, 1))

## markdown
## KPI 8: Watchlist Conversion Rate (target: above 40%)

## code
watched_count = watchlist['watched'].sum()
watchlist_conversion = watched_count / len(watchlist) * 100
print('Watchlist Conversion Rate:', round(watchlist_conversion, 1), '%')

if watchlist_conversion > 40:
    print('This meets the target of above 40%')
else:
    print('This does NOT meet the target of above 40%')

## markdown
## KPI 9: Hit Concentration
How much of all viewing comes from just the top 10% of titles by number of plays?

## code
plays_per_title = watch_history['title_id'].value_counts()
number_of_titles = len(plays_per_title)
top_10_percent_count = int(number_of_titles * 0.10)

top_titles_plays = plays_per_title.sort_values(ascending=False).head(top_10_percent_count)
hit_concentration = top_titles_plays.sum() / plays_per_title.sum() * 100

print('Number of titles in the top 10%:', top_10_percent_count)
print('Hit Concentration:', round(hit_concentration, 1), '%')

## markdown
## KPI 10: Originals Share of Watch Hours

## code
original_sessions = wh_titles[wh_titles['is_original'] == True]
originals_hours = original_sessions['watch_duration_min'].sum() / 60
originals_share = originals_hours / total_watch_hours * 100

print('Originals Watch Hours:', round(originals_hours))
print('Originals Share of Total Watch Hours:', round(originals_share, 1), '%')

## markdown
## KPI Summary Table

## code
kpi_table = pd.DataFrame({
    'KPI': ['Total Watch Hours', 'Active Rate', 'Churn Rate', 'Avg Completion Rate',
            'MRR', 'ARPU', 'Avg Watch Hours per Active Sub', 'Watchlist Conversion',
            'Hit Concentration (top 10% titles)', 'Originals Share of Hours'],
    'Value': [round(total_watch_hours), f'{round(active_rate,1)}%', f'{round(churn_rate,1)}%',
              f'{round(avg_completion,1)}%', f'${round(mrr,2)}', f'${round(arpu,2)}',
              round(avg_hours_per_subscriber,1), f'{round(watchlist_conversion,1)}%',
              f'{round(hit_concentration,1)}%', f'{round(originals_share,1)}%']
})
kpi_table

## markdown
## Question: Which subscriber segments are most valuable / engaged?
I'll look at watch hours per subscriber by plan type and by age group.

## code
wh_subs = watch_history.merge(subscribers, on='subscriber_id', how='left')
hours_by_plan = wh_subs.groupby('plan_type')['watch_duration_min'].sum() / 60
```

```
subs_by_plan = subscribers.groupby('plan_type').size()
hours_per_sub_by_plan = hours_by_plan / subs_by_plan

print('Watch hours per subscriber, by plan:')
print(hours_per_sub_by_plan.sort_values(ascending=False))

## code
# Put subscribers into age groups
def get_age_group(age):
    if age < 25:
        return '18-24'
    elif age < 35:
        return '25-34'
    elif age < 45:
        return '35-44'
    elif age < 55:
        return '45-54'
    elif age < 65:
        return '55-64'
    else:
        return '65+'

subscribers['age_group'] = subscribers['age'].apply(get_age_group)
wh_subs = watch_history.merge(subscribers[['subscriber_id','age_group']], on='subscriber_id', how='le

hours_by_age = wh_subs.groupby('age_group')['watch_duration_min'].sum() / 60
subs_by_age = subscribers.groupby('age_group').size()
hours_per_sub_by_age = hours_by_age / subs_by_age

print('Watch hours per subscriber, by age group:')
print(hours_per_sub_by_age.sort_values(ascending=False))

## markdown
**What this tells me:** Premium subscribers watch the most hours per person, and subscribers in their

## markdown
## Question: What does an at-risk (churned) subscriber look like?

## code
churn_by_plan = inactive_subscribers.groupby('plan_type').size() / subscribers.groupby('plan_type').s
print('Churn rate by plan:')
print(churn_by_plan.sort_values(ascending=False))

## code
print('Average tenure, churned subscribers:', round(inactive_subscribers['tenure_months'].mean(),1),
print('Average tenure, active subscribers:', round(active_subscribers['tenure_months'].mean(),1), 'mo

## markdown
**What this tells me:** Basic with Ads subscribers churn the most, and churned subscribers tend to ha

## markdown
## Question: Is content spend efficient? (watch hours per $1,000 spent, by genre)

## code
spend_by_genre = titles.groupby('primary_genre')['license_cost_usd'].sum()
hours_by_genre_catalogue = titles.groupby('primary_genre')['total_watch_hours'].sum()

efficiency = hours_by_genre_catalogue / (spend_by_genre / 1000)
efficiency = efficiency.sort_values(ascending=False)

print('Watch hours delivered per $1,000 spent, by genre:')
print(efficiency)

## markdown
**What this tells me:** Efficiency is not the same across genres - some genres return a lot more wate

## markdown
## Bonus: Cohort Retention
I'm grouping subscribers by the month they signed up (their "cohort"), and checking
what percent of each cohort was still active 3, 6, and 12 months after signing up.

## code
subscribers['signup_month'] = subscribers['signup_date'].dt.to_period('M')

# the most recent date anywhere in the watch history, used as "today" for this dataset
last_date = watch_history['watch_date'].max()
print('Using', last_date, 'as the reference date')
```

```
## code
cohort_list = subscribers['signup_month'].unique()
cohort_list = sorted(cohort_list)

results = []

for cohort in cohort_list:
    group = subscribers[subscribers['signup_month'] == cohort]
    if len(group) < 20:
        continue  # skip very small cohorts, not enough subscribers to trust the percentage

    row = {'cohort': str(cohort), 'subscribers': len(group)}

    for months in [3, 6, 12]:
        cutoff_date = group['signup_date'] + pd.DateOffset(months=months)
        # skip this cohort/horizon if not enough time has passed yet
        if (cutoff_date > last_date).all():
            row[f'retained_{months}m'] = None
            continue

        still_active_count = 0
        counted = 0
        for i in group.index:
            cutoff = group.loc[i, 'signup_date'] + pd.DateOffset(months=months)
            if cutoff > last_date:
                continue  # not enough time has passed for this subscriber yet
            counted += 1
            if group.loc[i, 'is_active']:
                still_active_count += 1
            elif pd.notna(group.loc[i, 'churn_date']) and group.loc[i, 'churn_date'] >= cutoff:
                still_active_count += 1

        if counted > 0:
            row[f'retained_{months}m'] = round(still_active_count / counted * 100, 1)
        else:
            row[f'retained_{months}m'] = None

    results.append(row)

cohort_retention = pd.DataFrame(results)
cohort_retention.tail(15)

## code
import matplotlib.pyplot as plt

plot_data = cohort_retention.dropna(subset=['retained_12m'])

plt.figure(figsize=(12,5))
plt.plot(plot_data['cohort'], plot_data['retained_3m'], marker='o', label='3-month retention')
plt.plot(plot_data['cohort'], plot_data['retained_6m'], marker='o', label='6-month retention')
plt.plot(plot_data['cohort'], plot_data['retained_12m'], marker='o', label='12-month retention')
plt.title('Cohort Retention by Signup Month')
plt.ylabel('% Still Active')
plt.xticks([])  # too many cohorts to show every label
plt.legend()
plt.show()

## markdown
**What this tells me:** 3-month retention is always the highest, and retention
drops off by 12 months, which is the normal shape for a subscription business. The gap
between 3-month and 12-month retention is a good number for leadership to track over
time.

## code
kpi_table.to_csv('kpi_summary.csv', index=False)
cohort_retention.to_csv('cohort_retention.csv', index=False)
print('Saved KPI summary and cohort retention files.')
```